

Trabajo práctico Nro 2

Filsinger Gonzalo, Perea Ignacio, Leon Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En este informe se abordará el desarrollo del trabajo práctico Nro 2, en el cual experimentaremos con las interrupciones externas del microcontrolador STM32F103C8T6, utilizando la placa BluePill. Se desarrollaran los primeros dos ejercicios del trabajo práctico.

Index Terms—BluePill, GPIO, LED, Memoria, Interrupciones, Botones.

I. INTRODUCCIÓN

EN este informe solamente se abordarán los primeros dos ejercicios del trabajo práctico, en el primero utilizaremos dos botones uno con mas prioridad que el otro para interrumpir el parpadeo de un led.

II. EJERCICIO 1: INTERRUPTIONES CON BOTONES DE BAJA Y ALTA PRIORIDAD

Para este programa se usará el código del TP1 como base, el cual se modificará para hacer interrupciones al led del TP1 con botones de baja y alta prioridad. Para conseguir esto se agregaron las siguientes definiciones en el archivo main.c:

- Se definió la dirección base **AFIO_BASE** con la dirección del registro AFIO.
- Se definió la dirección base **EXTI_BASE** con la dirección del registro EXTI, el cual se utiliza para configurar las interrupciones externas.
- Se definió la dirección base **NVIC_BASE** con la dirección del registro NVIC, el cual se utiliza para configurar las interrupciones en el NVIC.
- Se definió el registro **AFIO_EXTICR1** con la dirección del registro AFIO_EXTICR1, el cual se utiliza para configurar el pin del botón como fuente de interrupción externa.
- Se definió el registro **EXTI_IMR** con la dirección del registro EXTI_IMR, el cual se utiliza para habilitar las interrupciones de los pines configurados.
- Se definió el registro **EXTI_FTSR** con la dirección del registro EXTI_FTSR, el cual se utiliza para configurar la interrupción por flanco de bajada.
- Se definió el registro **EXTI_PR** con la dirección del registro EXTI_PR, el cual se utiliza para limpiar las banderas de interrupción.
- Se definió el registro **NVIC_ISER0** con la dirección del registro NVIC_ISER0, el cual se utiliza para habilitar las interrupciones en el NVIC.

- Se definió el registro **NVIC_IPR1** con la dirección del registro NVIC_IPR1, el cual se utiliza para configurar la prioridad de las interrupciones de los IRQ 4 a 7.
- Se definió el registro **NVIC_IPR2** con la dirección del registro NVIC_IPR2, el cual se utiliza para configurar la prioridad de las interrupciones de los IRQ 8 a 11.
- Se definieron las mascarar **GPIOA4** , **GPIOA5** y **GPIOA6**

```
#include <stdint.h>

// --- Direcciones Base ---
#define RCC_BASE 0x40021000
#define GPIOC_BASE 0x40011000
#define GPIOA_BASE 0x40010800
#define AFIO_BASE 0x40010000
#define EXTI_BASE 0x40010400
#define NVIC_BASE 0xE000E100

// --- Registros de Periféricos ---
#define RCC_APB2ENR *(volatile uint32_t*)(RCC_BASE + 0x18)
#define GPIOC_CRH *(volatile uint32_t*)(GPIOC_BASE + 0x04)
#define GPIOA_CRL *(volatile uint32_t*)(GPIOA_BASE + 0x00)
#define GPIOC_ODR *(volatile uint32_t*)(GPIOC_BASE + 0x0C)
#define GPIOA_ODR *(volatile uint32_t*)(GPIOA_BASE + 0x0C)

#define AFIO_EXTICR1 *(volatile uint32_t*)(AFIO_BASE + 0x08)
#define EXTI_IMR *(volatile uint32_t*)(EXTI_BASE + 0x00)
#define EXTI_FTSR *(volatile uint32_t*)(EXTI_BASE + 0x0C)
#define EXTI_PR *(volatile uint32_t*)(EXTI_BASE + 0x14)
#define NVIC_ISER0 *(volatile uint32_t*)(NVIC_BASE + 0x00)

#define NVIC_IPR1 *(volatile uint32_t*)(NVIC_BASE + 0x304)
#define NVIC_IPR2 *(volatile uint32_t*)(NVIC_BASE + 0x308)
#define SCB_SHPR3 *(volatile uint32_t*)(0xE000ED20)

// --- Campos de Bits ---
#define RCC_AFIOEN (1 << 0)
#define RCC_IOPAEN (1 << 2)
#define RCC_IOPCEN (1 << 4)
#define GPIOC13 (1UL << 13)
#define GPIOA6 (1UL << 6) // LED Botón Baja Prioridad
#define GPIOA5 (1UL << 5) // LED Botón Alta Prioridad
#define GPIOA4 (1UL << 4) // LED Normalidad en PA4
```

Definiciones de los registros en main.c

A continuación agregamos las funciones de interrupción para los botones, las cuales se encargan de encender y apagar el led correspondiente al botón presionado. Para el botón de baja prioridad (A1) se encenderá el led del pin A6, y para el botón de alta prioridad (A2) se encenderá el led del pin A5.

```
// Botón en PA1 (EXTI1) - PRIORIDAD BAJA -> Titila PA6 5 veces
void EXTI1_IRQHandler(void)
{
    if (EXTI_PR & (1 << 1))
    {
        EXTI_PR = (1 << 1);

        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA6;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA6;
    }
}

// Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila PA5 5 veces
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        EXTI_PR = (1 << 2);

        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA5;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA5;
    }
}
}
```

Agregado de funciones para interrupciones

Ahora pasamos a modificar la función **main**

```
// 1. RCC (Relojes)
RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN | RCC_IOPCEN;

// 2. GPIO
// PC13 (LED placa) como salida
GPIO_CRH &= 0xFF0FFFFF;
GPIO_CRH |= 0x00200000;

// Configurar PA6, PA5, PA4 (salidas) y PA2, PA1 (entradas)
GPIOA_CRL &= 0xF000F00F;
GPIOA_CRL |= 0x02220880;

// Activamos resistencias Pull-up internas en PA1 y PA2
GPIOA_ODR |= (1 << 1) | (1 << 2);

// 3. AFIO (Multiplexor)
AFIO_EXTICR1 &= ~(0x0000FF0);

// 4. EXTI
EXTI_IMR |= (1 << 1) | (1 << 2);
EXTI_FTSR |= (1 << 1) | (1 << 2);

// a) Garantizar que SysTick tenga la prioridad máxima (0x00)
SCB_SHPR3 &= ~(0xFF << 24);

// b) Limpiar las prioridades previas de EXTI1 (IRQ 7) y EXTI2 (IRQ 8)
NVIC_IPR1 &= ~(0xFF << 24); // Limpia bits 24 a 31
NVIC_IPR2 &= ~(0xFF << 0); // Limpia bits 0 a 7

// c) Asignar prioridades (recordando que el número menor es la prioridad más alta)
// Usamos los bits superiores de cada byte porque STM32 lee los 4 bits más significativos
NVIC_IPR2 |= (0x40 << 0); // PA2 (EXTI2) - ALTA Prioridad (0x40)
NVIC_IPR1 |= (0x80 << 24); // PA1 (EXTI1) - BAJA Prioridad (0x80)

// Jerarquía resultante: SysTick (0x00) > PA2 (0x40) > PA1 (0x80)

// 6. NVIC ENABLE
NVIC_ISER0 |= (1 << 7) | (1 << 8);

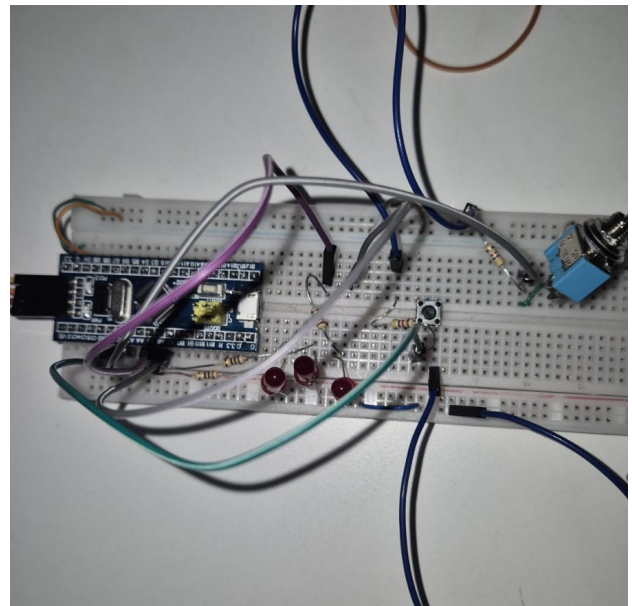
systick_init_ms();

while (1)
{
    // --- Tarea de Normalidad (Background Task) ---
    GPIOA_ODR ^= GPIOA4;
    GPIOC_ODR ^= GPIOC13;
    delay_ms_bloqueante(500);
}
```

Configuración de los pines y las interrupciones en main

Primero habilitamos los clocks del GPIO y AFIO, luego configuramos los pines A4, A5 y A6 como salida, y los A1 y A2 como entrada con pull-up. Luego asociamos el pin GPIO a la línea EXTI y pasamos a configurar las interrupciones como

flancos descendentes, por ultimo configuramos el NVIC con las prioridades correspondientes.



Resultado final del ejercicio 1

El funcionamiento es el siguiente: Al iniciar el led del pin A4 se mantiene en un loop constante de encender y apagar, si se interrumpe mediante el botón de baja prioridad (A1) el led del pin A6 se enciende y el del pin A4 se apaga, si se interrumpe mediante el botón de alta prioridad (A2) el led del pin A5 se enciende y el del pin A4 se apaga. Si mediante la interrupcion de baja prioridad se interrumpe con alta prioridad, el led del pin A5 se enciende y el del pin A6 se apaga, hasta terminar su ciclo de 5 encendidos y apagados, y luego el led A6 termina su ciclo donde lo dejo para luego volver al loop principal.

III. CONCLUSIÓN

Se logró realizar el ejercicio 1, utilizando las interrupciones externas del microcontrolador para controlar el encendido de los leds mediante botones de baja y alta prioridad.

IV. CODIGOS

IV-A. main.c(Ejercicio 1)

```

1 #include <stdint.h>
2
3 // --- Direcciones Base ---
4 #define RCC_BASE 0x40021000
5 #define GPIOC_BASE 0x40011000
6 #define GPIOA_BASE 0x40010800
7 #define AFIO_BASE 0x40010000
8 #define EXTI_BASE 0x40010400
9 #define NVIC_BASE 0xE000E100
10
11 // --- Registros de Periféricos ---
12 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
13     + 0x18)
14 #define GPIOC_CRH *(volatile uint32_t *) (
15     GPIOC_BASE + 0x04)
16 #define GPIOA_CRL *(volatile uint32_t *) (
17     GPIOA_BASE + 0x00)
18 #define GPIOC_ODR *(volatile uint32_t *) (
19     GPIOC_BASE + 0x0C)
20 #define GPIOA_ODR *(volatile uint32_t *) (
21     GPIOA_BASE + 0x0C)
22
23 #define AFIO_EXTICR1 *(volatile uint32_t *) (
24     AFIO_BASE + 0x08)
25 #define EXTI_IMR *(volatile uint32_t *) (
26     EXTI_BASE + 0x00)
27 #define EXTI_FTSR *(volatile uint32_t *) (
28     EXTI_BASE + 0x0C)
29 #define EXTI_PR *(volatile uint32_t *) (
30     EXTI_BASE + 0x14)
31 #define NVIC_ISER0 *(volatile uint32_t *) (
32     NVIC_BASE + 0x00)
33 #define NVIC_IPR1 *(volatile uint32_t *) (
34     NVIC_BASE + 0x304) // Controla IRQ 4 a 7
35 #define NVIC_IPR2 *(volatile uint32_t *) (
36     NVIC_BASE + 0x308) // Controla IRQ 8 a 11
37 #define SCB_SHPR3 *(volatile uint32_t *) (0
38     xE000ED20) // Controla prioridad del
39     SysTick
40
41 // --- Campos de Bits ---
42 #define RCC_AFIOEN (1 << 0)
43 #define RCC_IOPAEN (1 << 2)
44 #define RCC_IOPCEN (1 << 4)
45 #define GPIOC13 (1UL << 13)
46 #define GPIOA6 (1UL << 6) // LED Botón Baja
47     Prioridad
48 #define GPIOA5 (1UL << 5) // LED Botón Alta
49     Prioridad
50 #define GPIOA4 (1UL << 4) // LED Normalidad en
51     PA4
52
53 // --- Registros del SysTick ---
54 #define SysTick_BASE 0xE000E010
55 #define SysTick_CTRL *(volatile uint32_t *) (
56     SysTick_BASE + 0x00)
57 #define SysTick_LOAD *(volatile uint32_t *) (
58     SysTick_BASE + 0x04)
59 #define SysTick_VAL *(volatile uint32_t *) (
60     SysTick_BASE + 0x08)
61
62 #define SysTick_CTRL_ENABLE (1 << 0)
63 #define SysTick_CTRL_TICKINT (1 << 1)
64 #define SysTick_CTRL_CLKSOURCE (1 << 2)
65
66 volatile uint32_t tick;
67
68 // --- Funciones de Tiempo ---
69 void systick_init_ms(void)
70 {
71     tick = 0;
72     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE;
73
74     SysTick_LOAD = 999;
75     SysTick_VAL = 0;
76     SysTick_CTRL |= SysTick_CTRL_TICKINT |
77         SysTick_CTRL_ENABLE;
78 }
79
80 void delay_ms_bloqueante(uint32_t ms)
81 {
82     uint32_t tiempo_inicio = tick;
83     while ((tick - tiempo_inicio) < ms);
84 }
85
86 // --- Rutinas de Servicio de Interrupción (ISR) ---
87 void SysTick_Handler(void)
88 {
89     tick++;
90 }
91
92 void EXTI1_IRQHandler(void)
93 {
94     if (EXTI_PR & (1 << 1))
95     {
96         EXTI_PR = (1 << 1);
97
98         for (int i = 0; i < 10; i++)
99         {
100             GPIOA_ODR ^= GPIOA6;
101             delay_ms_bloqueante(200);
102         }
103
104         GPIOA_ODR &= ~GPIOA6;
105     }
106 }
107
108 // Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila
109     PA5 5 veces
110 void EXTI2_IRQHandler(void)
111 {
112     if (EXTI_PR & (1 << 2))
113     {
114         EXTI_PR = (1 << 2);
115
116         for (int i = 0; i < 10; i++)
117         {
118             GPIOA_ODR ^= GPIOA5;
119             delay_ms_bloqueante(200);
120         }
121
122         GPIOA_ODR &= ~GPIOA5;
123     }
124 }
125
126 // --- Función Principal ---
127 void main(void)
128 {
129     // 1. RCC (Relojes)
130     RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN |
131         RCC_IOPCEN;
132
133     // 2. GPIO
134     // PC13 (LED placa) como salida
135     GPIOC_CRH &= 0xFFFFFFF;
136     GPIOC_CRH |= 0x00200000;
137
138     // Configurar PA6, PA5, PA4 (salidas) y PA2, PA1
139     // (entradas)
140     GPIOA_CRL &= 0xF000F00F;
141     GPIOA_CRL |= 0x02220880;
142
143     // Activamos resistencias Pull-up internas en
144     // PA1 y PA2
145     GPIOA_ODR |= (1 << 1) | (1 << 2);
146
147     // 3. AFIO (Multiplexor)
148     AFIO_EXTICR1 &= ~(0x00000FF0);

```

```

126 // 4. EXTI
127 EXTI_IMR |= (1 << 1) | (1 << 2);
128 EXTI_FTSR |= (1 << 1) | (1 << 2);
129
130 // 5. NVIC - Configuración de Prioridades
131 // a) Garantizar que SysTick tenga la prioridad
132 // máxima (0x00)
133 SCB_SHPR3 &= ~(0xFF << 24);
134
135 // b) Limpiar las prioridades previas de EXTI1 (
136 // IRQ 7) y EXTI2 (IRQ 8)
137 NVIC_IPR1 &= ~(0xFF << 24); // Limpia bits 24 a
138 // 31
139 NVIC_IPR2 &= ~(0xFF << 0); // Limpia bits 0 a 7
140
141 // c) Asignar prioridades (recordando que el nú
142 // mero menor es la prioridad más alta)
143 // Usamos los bits superiores de cada byte
144 // porque STM32 lee los 4 bits más significativos
145 NVIC_IPR2 |= (0x40 << 0); // PA2 (EXTI2) - ALTA
146 // Prioridad (0x40)
147 NVIC_IPR1 |= (0x80 << 24); // PA1 (EXTI1) - BAJA
148 // Prioridad (0x80)
149
150 // Jerarquía resultante: SysTick (0x00) > PA2 (0
151 // x40) > PA1 (0x80)
152
153 // 6. NVIC ENABLE
154 NVIC_ISER0 |= (1 << 7) | (1 << 8);
155
156 systick_init_ms();
157
158 while (1)
159 {
160     // --- Tarea de Normalidad (Background Task)
161     ---
162     GPIOA_ODR ^= GPIOA4;
163     GPIOC_ODR ^= GPIOC13;
164     delay_ms_bloqueante(500);
165 }

```

Listing 1. Configuración inicial en main.c

IV-B. crt.s(Ejercicio 1)

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
7 // text.manejadores
8 // función básica para manejar los punteros
9 .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b . // Bucle infinito */
13
14 // definir los vectores ejemplo
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak SysTick_Handler
18 .thumb_set SysTick_Handler, Default_Handler
19 .weak HardFault_Handler
20 .thumb_set HardFault_Handler, Default_Handler
21 .weak WWDG_IRQHandler
22 .thumb_set WWDG_IRQHandler, Default_Handler
23 .weak PVD_IRQHandler
24 .thumb_set PVD_IRQHandler, Default_Handler
25 .weak TAMPER_IRQHandler
26 .thumb_set TAMPER_IRQHandler, Default_Handler
27 .weak RTC_IRQHandler
28 .thumb_set RTC_IRQHandler, Default_Handler
29 .weak FLASH_IRQHandler
30 .thumb_set FLASH_IRQHandler, Default_Handler
31 .weak RCC_IRQHandler
32 .thumb_set RCC_IRQHandler, Default_Handler
33 .weak EXTI0_IRQHandler
34 .thumb_set EXTI0_IRQHandler, Default_Handler
35 .weak EXTI1_IRQHandler
36 .thumb_set EXTI1_IRQHandler, Default_Handler
37 .weak EXTI2_IRQHandler
38 .thumb_set EXTI2_IRQHandler, Default_Handler
39 .weak EXTI3_IRQHandler
40 .thumb_set EXTI3_IRQHandler, Default_Handler
41 .weak EXTI4_IRQHandler
42 .thumb_set EXTI4_IRQHandler, Default_Handler
43
44 // vectores
45 .section .isr_vector,"a",%progbits
46 .word _esstack
47 .word _reset
48 .word NMI_Handler // 2: NMI (Non-Maskable
49 // Interrupt) */
50 .word HardFault_Handler // 3: HardFault (falla
51 // grave) */
52 .word 0 // 4: MemManage (falla de
53 // memoria) */
54 .word 0 // 5: BusFault (falla de
55 // bus) */
56 .word 0 // 6: UsageFault (falla
57 // de uso) */
58 .word 0 // 7: Reservado */
59 .word 0 // 8: Reservado */
60 .word 0 // 9: Reservado */
61 .word 0 // 10: Reservado */
62 .word 0 // 11: SVCALL (Llamada al
63 // Supervisor) */
64 .word 0 // 12: Debug Monitor */
65 .word 0 // 13: Reservado */
66 .word 0 // 14: PendSV (Interrupció
67 // n pendiente del sistema) */
68 .word SysTick_Handler // 15: El handler del
69 // SysTick */
70 .word WWDG_IRQHandler
71 .word PVD_IRQHandler
72 .word TAMPER_IRQHandler
73 .word RTC_IRQHandler
74 .word FLASH_IRQHandler
75 .word RCC_IRQHandler
76 .word EXTI0_IRQHandler
77 .word EXTI1_IRQHandler
78 .word EXTI2_IRQHandler
79 .word EXTI3_IRQHandler
80 .word EXTI4_IRQHandler
81
82 // declaramos la función _reset dentro de la seccion
83 // text.reset
84 .section .text.reset
85 .thumb_func
86 .global _reset
87 _reset:
88     b1 main
89     b .

```